Navee

Theme System Reference

Custom Theme Guide

Abstract

Custom themes replace the fold area (the top section of the main dashboard) with your own HTML layout. You supply HTML for the structure and CSS for the look; the app fills your elements with live scooter data automatically, every 500 ms, with no JavaScript on your side. This document is the complete reference: the bundle format, every live-data hook, the live-state signals, the circular speed-gauge technique, the CSS variables, the security model and a full worked example.

Author Laufbursche42

Guide version 1.0

Repository github.com/Laufbursche42/nv-lb-edition

Contents

1 What a theme is and is not **2**

2 The bundle format **2**

3 Theme scope **2**
3.1 Full-scope DOM skeleton . . . 3

4 theme.html - structure and hooks **3**
4.1 Live value hooks - data-lb . . . 3
4.2 State hooks - data-lb-state . . . 4
4.3 Action hooks - data-lb-action . . . 4
4.4 Tile registration - data-tile-id . . . 4
4.5 Available data keys . . . 4

5 Live-state signals on `<html>` **5**
5.1 Throttle-lock state . . . 5
5.2 Display unit . . . 6

6 Circular speed gauge - complete technique **6**
6.1 Geometry . . . 6
6.2 Animating the fill arc with --cur-speed . . . 6
6.3 Managed scale marks (optional) - data-lb-gauge . . . 7
6.4 Unit label toggle . . . 7
6.5 Inline SVG icons . . . 8
6.6 Complete minimal gauge . . . 8

7 theme.css - layout and variables **9**
7.1 App-provided CSS variables . . . 9

8 theme.json - metadata **12**

9 CSS variable overrides - cssVars **12**
9.1 Label localization . . . 13

10 Security model - no JavaScript **13**

11 How to import **14**
11.1 Step 1 - Create the ZIP . . . 14
11.2 Step 2 - Import . . . 14

12 Portability between Laufbursche apps **14**

13 Translations and language **15**
13.1 Using existing app strings . . . 15
13.2 Adding your own translated strings . . . 15

14 Complete example - Orbit (full-scope teaching theme) **16**
14.1 theme.json . . . 16
14.2 theme.html . . . 16

14.3 theme.css 18

15 Tips and limitations **20**

1 What a theme is and is not

A custom theme **replaces the HTML inside `#fold-inner`** - the part of the main screen that sits above the scroll area. You control layout, typography and colours with your own HTML and CSS. The app keeps driving everything else: the scroll tiles below, the quick-action button row, the recording banner and all the sub-pages.

A theme is not a web app. It cannot run JavaScript, load external resources or change the app's logic. It is structure and style only, with data fed in by the app.

---

2 The bundle format

A theme is a **ZIP file** containing exactly three files at the root level of the archive:

File	Required	Purpose
`theme.html`	**Yes**	HTML injected into the dashboard (see Theme scope below)
`theme.css`	**Yes**	CSS loaded for this theme (scoped to your HTML)
`theme.json`	No	Metadata: display name, version, scope, optional CSS-variable declarations

The ZIP filename becomes the import fallback name if `theme.json` has no `name` field.

Size limit: 2 MB per file.

---

3 Theme scope

The `scope` field in `theme.json` controls how much of the dashboard your theme replaces.

`scope` value	Injection target	Behavior
`"fold"` (default, or omit the field)	`#fold-inner`	Replaces only the hero section at the top of the screen. The
`"full"`	`#scroller`	Replaces the **entire dashboard** - both the hero section an

fold scope is best for lightweight reskins: you redesign the top hero area and let the standard tiles handle data display below.

full scope is for full dashboard redesigns: your `theme.html` must include both the hero section and any data tiles you want shown. All `data-lb`, `data-lb-state`, and `data-lb-action` hooks work identically in both scopes.

Example `theme.json` for a full-scope theme:

```
{
  "name": "My Dashboard",
  "scope": "full",
```

```
  "version": "1.0",
  "author": "you"
}
```

A full-scope `theme.html` should structure its content as sections flowing top-to-bottom, each registered with `data-tile-id` so the tile manager can show, hide and reorder them.

3.1 Full-scope DOM skeleton

There is no required outer wrapper. Sections flow in document order:

```
<!-- No outer wrapper needed - sections flow in document order. -->

<!-- Section 1: hero / gauge area (top of screen) -->
<div data-tile-id="my-hero" data-tile-label="Tacho">
  <!-- hero content, SVG gauge, etc. -->
</div>

<!-- Section 2: data tile grid -->
<div data-tile-id="my-grid" data-tile-label="Daten">
  <!-- data cells with data-lb keys -->
</div>

<!-- Section 3: quick-action button row (bottom) -->
<div data-tile-id="my-buttons" data-tile-label="Buttons">
  <!-- action buttons with data-lb-action -->
</div>
```

Rules for full scope:

- Each direct child (or any descendant) can carry `data-tile-id`.
- The tile manager lists each registered section by its `data-tile-label`.
- Sections without `data-tile-id` are always visible and cannot be managed.
- The built-in scroll tiles (`#scroller` content) are gone for the duration of the theme; your sections provide all data display.

---

4 theme.html - structure and hooks

`theme.html` is a plain HTML fragment - **no `<html>`, `<head>` or `<body>` tags**, just the content you want inside `#fold-inner` (fold scope, the default) or `#scroller` (full scope - see Theme scope). The app sanitizes it before storing (see Security model).

4.1 Live value hooks - data-lb

Add `data-lb="key"` to any element. The app sets that element's `textContent` to the current scooter value every 500 ms:

```
<div class="my-speed" data-lb="speed"></div>
<div class="my-soc"   data-lb="soc"></div>
```

Values are numbers (or `true`/`false` for booleans). Format them with CSS (`content` on `::after` for units, or just text next to the value). If the scooter is disconnected, the last known value stays until the next connection.

4.2 State hooks - data-lb-state

Add `data-lb-state="key:classname"` to toggle a CSS class based on whether a data value is truthy:

```
<div class="speed-box" data-lb-state="park:is-parked">
```

When `data.park` is `true`, the class `is-parked` is added; when `false`, it is removed. Use this for visual state changes (different background when parked, flashing when locked, etc.).

4.3 Action hooks - data-lb-action

One built-in action is supported:

Value	Behaviour
`speed-unlock`	Triple-tap this element to trigger the eKFV speed-unlock sequence (same as the default speed tile

```
<div class="my-speed" data-lb="speed" data-lb-action="speed-unlock"></div>
```

4.4 Tile registration - data-tile-id

Wrap sections of your theme in an element with `data-tile-id` and `data-tile-label` to register them with the tile manager. Registered tiles appear in the theme manager's tile list, can be shown or hidden, and can be drag-reordered by the user:

```
<div data-tile-id="my-speed-block" data-tile-label="Tacho">
  <!-- speed content -->
</div>
<div data-tile-id="my-motor-block" data-tile-label="Motoren">
  <!-- motor content -->
</div>
```

Without `data-tile-id`, a section is always shown and cannot be managed.

4.5 Available data keys

All keys come from the scooter's BLE telemetry, parsed every 500 ms. Raw numbers are `0` when the scooter is disconnected; `park` is `false`.

Key	Type	Unit	Description
`speed`	number	km/h or mph	Current speed
`avgSpeed`	number	km/h or mph	Average speed (this ride)
`maxSpeed`	number	km/h or mph	Maximum speed (this ride)
`soc`	number	%	State of charge
`soh`	number	%	State of health
`volt`	number	V	Pack voltage
`poleVolt`	number	V	Pole voltage
`current`	number	A	Pack current (null when not reported)

Key	Type	Unit	Description
`gear`	number	-	Active gear (1-3)
`rearCurrent`	number	A	Rear motor current
`frontCurrent`	number	A	Front motor current
`rTemp`	number	°C	Rear motor temperature
`fTemp`	number	°C	Front motor temperature
`power`	number	kW	Total power
`singleMile`	number	km or mi	Trip distance
`totalMile`	number	km or mi	Odometer
`fb`	number	-	Feedback/regen value
`rssi`	number	dBm	Bluetooth signal strength
`park`	boolean	-	`true` when scooter is in park
`vcuUnlock`	number	-	`0` = locked, `1` = unlocked, `-1` = unknown

Unit note: `speed`, `avgSpeed`, `maxSpeed`, `singleMile` and `totalMile` automatically follow the app's km/mph setting - the number you receive is already in the user's chosen unit.

Value format: all numbers are raw JavaScript floats, e.g. `speed` = `25.3`, `power` = `0.24`. There is no formatting control without JavaScript. If you need an integer display, CSS cannot round numbers; design your layout so a one-decimal value fits (wider container, or simply accept the precision the scooter reports).

---

5 Live-state signals on `<html>`

The app sets classes and CSS variables directly on the `<html>` element so script-free theme CSS can react to live state without needing a `data-lb-state` attribute on a specific element.

5.1 Throttle-lock state

Signal	Type	Values	When set
`lb-unlocked`	class on `<html>`	present / absent	Present when the scooter is de-restricted (unlocke
`--throttle-unlocked`	CSS var on `:root`	`1` / `0`	`1` = unlocked, `0` = locked; useful in `calc()`

Updated every BLE tick (500 ms). Defaults to absent / `0` at page load.

CSS usage:

```css
/* Scale ring shows full 0-100 range when unlocked */
.gauge-scale { --scale-max: 40; }
:root.lb-unlocked .gauge-scale { --scale-max: 100; }

/* Or with the numeric var */
.gauge-arc {
  stroke-dasharray: calc(var(--throttle-unlocked) * 220 + 80);
}
```

5.2 Display unit

Signal	Type	Values	When set
`lb-mph`	class on `<html>`	present / absent	Present when the user has selected mph; absent for km/h

Set at startup and whenever the user toggles the unit in settings.

CSS usage:

```
.speed-unit::after { content: 'km/h'; }
:root.lb-mph .speed-unit::after { content: 'mph'; }
```

---

6 Circular speed gauge - complete technique

A circular speed gauge is the most common full-scope theme element. This section gives you the exact SVG markup and CSS math to build one from scratch.

6.1 Geometry

The gauge uses two stacked `<circle>` SVG elements:

- **Track ring** - full 270-degree grey background arc.
- **Fill arc** - shorter arc that grows with `--cur-speed`.

The technique is `stroke-dasharray` / `stroke-dashoffset`:

```
stroke-dasharray="DRAW GAP"
```

- `DRAW` = length of the visible segment (SVG user units).
- `GAP` = gap that fills the rest of the circle.

For radius 58 in a 160×160 viewBox:

- Circumference = 2 × × 58 364 px
- 270° = 75% of 364 **273 px** (draw)
- Gap = 364 - 273 = **91 px**

So: `stroke-dasharray="273 91"` draws a 270° arc.

By default the arc starts at 3 o'clock (SVG zero-angle). Rotate 135° around the centre to start at 7 o'clock:

```
transform="rotate(135 80 80)"
```

6.2 Animating the fill arc with --cur-speed

Control the fill arc's visible length with `stroke-dashoffset`:

- `dashoffset = 0` → full 273 px visible (maximum speed).
- `dashoffset = 273` → arc fully hidden (zero speed).
- Formula: `offset = 273 - speed × 273 / maxSpeed`

In CSS, define `--speed-max` on the element and reference `--cur-speed` from `:root`:

```css
.gauge-arc {
  --speed-max: 25;   /* eKFV locked: 25 km/h */
  stroke-dashoffset: calc(273 - var(--cur-speed) * 273 / var(--speed-max));
  transition: stroke-dashoffset 0.4s ease-out;
}

/* Extend scale when the scooter is unlocked (lb-unlocked on <html>). */
:root.lb-unlocked .gauge-arc { --speed-max: 100; }
```

No JavaScript needed - `--cur-speed` is updated by the app every 500 ms and CSS recalculates automatically.

6.3 Managed scale marks (optional) - data-lb-gauge

The built-in Carbon and Lambo themes let the rider choose the gauge's number scale - **Klar**, **Fine** or **Round** - under **Settings -> Theme manager -> "Speedometer scale"**. The app draws the tick marks and number labels (evenly spaced, so dense low values like 20 / 22 / 25 stay readable) and drives the needle so it points exactly at the chosen number.

Your own full-scope theme can opt into this instead of drawing the scale by hand: add a `data-lb-gauge` attribute to your gauge `<svg>` and leave two empty groups for the app to fill.

```html
<svg class="my-gauge"
     data-lb-gauge='{"prefix":"my","cx":100,"cy":100,"labelR":61,"tickR0":84,"tickR1":94,"arcR":
     viewBox="0 0 200 200" aria-hidden="true">
  <!-- your static track ring, pointer, etc. -->
  <g class="my-scale scale-locked"></g>
  <g class="my-scale scale-unlocked"></g>
</svg>
```

Field	Meaning
`prefix`	class prefix for the elements the app injects (`my-tick`, `my-lbl`, `my-redzone-hi`) - style them in
`cx` / `cy`	gauge centre, in viewBox units
`labelR`	radius of the number labels
`tickR0` / `tickR1`	inner / outer radius of each tick line
`arcR`	radius of the red-zone arc
`arcPct`	percent of the full circle the 270-degree arc spans (use `75`)

The app fills `.my-scale.scale-locked` with the locked marks and `.scale-unlocked` with the open marks (km/h in `.lbl-kmh`, mph in `.lbl-mph`, toggled by `lb-mph`), adds a `.my-redzone-hi` arc, and sets `--cn-f` (0-1, the piecewise needle fraction) on the `<svg>` every tick. Drive your pointer and fill arc from it, e.g. `transform: rotate(calc(135deg + var(--cn-f) * 270deg))`. Leave `data-lb-gauge` off to keep the fully manual pure-CSS scale shown above.

6.4 Unit label toggle

Use two overlapping `<text>` elements toggled by `opacity`. `display:none` is unreliable on SVG text in Android WebView.

```
<text class="g-unit g-unit-kmh" x="80" y="100" text-anchor="middle">km/h</text>
<text class="g-unit g-unit-mph" x="80" y="100" text-anchor="middle">mph</text>
```

```
.g-unit-kmh { opacity: 1; }
.g-unit-mph { opacity: 0; }
:root.lb-mph .g-unit-kmh { opacity: 0; }
:root.lb-mph .g-unit-mph { opacity: 1; }
```

6.5 Inline SVG icons

No `url()` or `http://` sources are allowed. Embed SVG directly:

```
<svg class="my-icon" viewBox="0 0 24 24" fill="currentColor" aria-hidden="true">
  <path d="M7 2v11h3v9l7-12h-4l4-8z"/>   <!-- lightning bolt -->
</svg>
```

`fill="currentColor"` inherits the CSS `color` property, so tinting is just `color: var(--value)`.

6.6 Complete minimal gauge

```
<svg class="g-gauge" viewBox="0 0 160 160" aria-hidden="true">
  <!-- Track ring: 270° static background arc.
       rotate(135 80 80) puts the start at 7 o'clock. -->
  <circle class="g-track"
    cx="80" cy="80" r="58" fill="none"
    stroke-dasharray="273 91"
    transform="rotate(135 80 80)" />
  <!-- Fill arc: stroke-dashoffset driven by --cur-speed in CSS. -->
  <circle class="g-arc"
    cx="80" cy="80" r="58" fill="none"
    stroke-dasharray="273 91"
    transform="rotate(135 80 80)" />
  <text class="g-val" x="80" y="80" text-anchor="middle" dominant-baseline="middle"
        data-lb="speed">0</text>
  <text class="g-unit g-unit-kmh" x="80" y="100" text-anchor="middle">km/h</text>
  <text class="g-unit g-unit-mph" x="80" y="100" text-anchor="middle">mph</text>
</svg>
```

```
.g-gauge { width: 100%; max-width: 200px; overflow: visible; }

.g-track {
  stroke: var(--border);
  stroke-width: 10;
  stroke-linecap: round;
}

.g-arc {
  --speed-max: 25;
  stroke: var(--value);
  stroke-width: 10;
  stroke-linecap: round;
```

```
  stroke-dashoffset: calc(273 - var(--cur-speed) * 273 / var(--speed-max));
  transition: stroke-dashoffset 0.4s ease-out;
}
:root.lb-unlocked .g-arc { --speed-max: 100; }

.g-val       { font-size: 38px; font-weight: 700; fill: var(--text); font-family: inherit; }
.g-unit      { font-size: 12px; fill: var(--label); font-family: inherit; }
.g-unit-kmh  { opacity: 1; }
.g-unit-mph  { opacity: 0; }
:root.lb-mph .g-unit-kmh { opacity: 0; }
:root.lb-mph .g-unit-mph { opacity: 1; }
```

---

7 theme.css - layout and variables

`theme.css` is loaded as a scoped stylesheet after the app's own CSS. Write selectors that target your own class names, or element IDs you defined in `theme.html`.

You do **not** need to set colours or fonts from scratch. Use the app's CSS variables and your theme automatically adapts to dark mode, light mode and any per-theme style the user has configured.

7.1 App-provided CSS variables

These variables are always defined on `:root` by the app:

Runtime colour tokens - use these in `theme.css`

These are the mode-resolved tokens. They switch between the dark/light values automatically as the user changes mode. Always use these in selectors and `var()` references in your CSS.

Variable	Description
`--card`	Tile / card background
`--bg`	Page background
`--text`	Primary text
`--label`	Secondary / muted text
`--border`	Border / divider colour
`--value`	Accent / highlight colour (user-configurable)
`--ble`	BLE speed indicator colour
`--gps`	GPS speed indicator colour
`--good`	Positive / OK state colour
`--warn`	Warning colour
`--danger`	Error / critical colour
`--unlock`	"Unlocked / entdrosselt" indicator colour
`--btnfill`	Button fill colour

Per-mode source tokens - declare these in `cssVars` to customise the palette

These are the per-mode values that feed the runtime tokens above. Declare them in `cssVars` in `theme.json` to set your theme's own palette. The Style Editor exposes them grouped by mode.

Variable	Default (dark)	Default (light)	Description
`--dark-card`	`#111420`		Tile background in dark mode
`--dark-bg`	`#0A0C10`		Page background in dark mode
`--dark-border`	`#1C2035`		Border colour in dark mode
`--dark-text`	`#E4EDF6`		Primary text in dark mode
`--dark-label`	`#C6D2E2`		Secondary text in dark mode
`--dark-lum`	`90`		Brightness 0-100 in dark mode
`--light-card`		`#ffffff`	Tile background in light mode
`--light-bg`		`#e8edf5`	Page background in light mode
`--light-border`		`#CDD5E4`	Border colour in light mode
`--light-text`		`#0A0D18`	Primary text in light mode
`--light-label`		`#0A0D18`	Secondary text in light mode
`--light-lum`		`100`	Brightness 0-100 in light mode

Accent colour defaults

Declare these in `cssVars` to set the theme's default accent palette. Users can override them in the Style Editor.

Variable	Default	Description
`--value`	`#BCEB39`	Primary accent / value colour
`--ble`	`#B8F020`	BLE speed indicator
`--gps`	`#3DB8FF`	GPS speed indicator
`--good`	`#1EE89A`	Positive state colour
`--btnfill`	`#B8F020`	Button fill
`--warn`	`#FFAA18`	Warning colour
`--danger`	`#FF5050`	Error / critical colour
`--unlock`	`#FF5050`	Unlocked indicator colour

Typography - fixed scale, not editable

Variable	Default	Description
`--fs-2xs`	10 px	Smallest text (badges, tags)
`--fs-xs`	11 px	Extra-small text
`--fs-sm`	12 px	Small text
`--fs-md`	13 px	Medium text
`--fs-base`	14 px	Base body text
`--fs-xl`	16 px	Large text
`--fw-semibold`	600	Semibold weight
`--fw-bold`	700	Bold weight

Tile element font sizes (user-configurable via Style Editor; declare in `cssVars` to set theme defaults)

Variable	Default	Description
`--sz-speed-val`	64 px	Speed number
`--sz-speed-lbl`	14 px	Speed label
`--sz-hero-val`	38 px	Hero tile value
`--sz-hero-lbl`	13 px	Hero tile label
`--sz-m1-top`	12 px	Motor tile 1 title
`--sz-m1-mid`	32 px	Motor tile 1 value
`--sz-m1-bot`	12 px	Motor tile 1 unit
`--sz-m2-top`	12 px	Motor tile 2 title
`--sz-m2-mid`	32 px	Motor tile 2 value
`--sz-m2-bot`	12 px	Motor tile 2 unit

Tile minimum heights (user-configurable; `0` = natural content height)

Variable	Default	Description
`--tile-h-speed`	0 px	Speed tile minimum height
`--tile-h-hero`	0 px	Hero tile minimum height
`--tile-h-motors`	0 px	Motor tile minimum height

Transparency (user-configurable via Style Editor; always set for every theme)

The Style Editor shows "Transparenz / Durchsichtigkeit" sliders. **Slider scale: 0 % = fully solid (default), 100 % = fully transparent.** The injected CSS variable is always **opacity** (1.0 = solid, 0.0 = invisible); the app converts internally with `opacity = 1 - slider / 100`.

Variable	CSS value	Default	Description
`--tile-alpha`	0.0-1.0	`1.0`	Tile / card opacity. 1.0 = fully opaque, 0.0 = invisible.
`--btn-alpha`	0.0-1.0	`1.0`	Button opacity. 1.0 = fully opaque, 0.0 = invisible.
`--gauge-alpha`	0.0-1.0	`1.0`	Gauge speed-fill arc opacity. 1.0 = fully opaque, 0.0 = invisible.

Usage in `theme.css`:

```
/* Simple: apply opacity to the whole element (affects text too) */
[data-tile-id] { opacity: var(--tile-alpha); }
.lb-btn        { opacity: var(--btn-alpha);  }
.gauge-arc     { opacity: var(--gauge-alpha); }

/* Background / fill transparency only (CSS Color Level 4 relative colour syntax,
   Chromium 111+ / Android WebView 111+): */
[data-tile-id] { background-color: rgb(from var(--card)    r g b / var(--tile-alpha)); }
.lb-btn        { background-color: rgb(from var(--btnfill) r g b / var(--btn-alpha)); }
.gauge-arc     { fill:             rgb(from var(--value)   r g b / var(--gauge-alpha)); }
```

> **Gotcha:** `rgb(from color-mix(...) r g b / alpha)` does **not** work on Android WebView - a `color-mix()` origin silently drops the alpha channel. Always use a **flat CSS variable** as the origin (e.g. `var(--card)`, `var(--value)`, `var(--btnfill)`), never a `color-mix()` result. This applies to all three alpha vars.

Live telemetry

Variable	Unit	Description
`--cur-speed`	km/h (always)	Current speed, raw km/h regardless of the display-unit setting. Updated every Bl

Shape

Variable	Default	Description
`--radius`	12 px	Standard border radius

---

8 theme.json - metadata

Optional. If present, it must be valid JSON:

```
{
  "name": "My Theme",
  "version": "1.0"
}
```

Field	Description
`name`	Display name shown in the app's theme list. Falls back to the ZIP filename if absent.
`version`	Free-form version string, informational only.
`scope`	`"fold"` (default) or `"full"`. How much of the dashboard the theme replaces. See Theme scope.
`cssVars`	Object of CSS variable overrides. These become the theme's default palette and are shown in the Style
`strings`	Translation table for custom i18n keys. See Translations and language.

---

9 CSS variable overrides - cssVars

Declare `cssVars` in `theme.json` to set your theme's default colour / size palette. The values load into the Style Editor as the starting point before any user override. Merge order: **user-saved override > theme `cssVars` default > app built-in default.**

Two formats are accepted:

Flat string (minimal - value only):

```
{
  "name": "My Theme",
  "cssVars": {
    "--value": "#F4B223",
    "--dark-card": "#1A1500",
    "--dark-bg": "#0D0F00"
  }
}
```

Object format (preferred - shown in Style Editor with labels and sections):

```
{
  "name": "My Theme",
  "cssVars": {
    "--value":      { "default": "#F4B223", "label": "Accent",      "group": "accents" },
    "--dark-card":  { "default": "#1A1500", "label": "Kachel-HG",   "group": "dark"    },
    "--dark-bg":    { "default": "#0D0F00", "label": "Hintergrund", "group": "dark"    },
    "--light-card": { "default": "#FFF8E0", "label": "Kachel-HG",   "group": "light"   },
    "--light-bg":   { "default": "#F5EFD0", "label": "Hintergrund", "group": "light"   }
  }
}
```

Valid `group` values that map to Style Editor sections: `"dark"`, `"light"`, `"accents"`, `"fonts"`, `"tiles"`. Any variable you list in `cssVars` can be further overridden by the user in the Style Editor. Declare your full custom palette in `cssVars` to expose it for user tweaking.

9.1 Label localization

The `label` field in a `cssVars` entry accepts three formats:

1. i18n key string - resolved via the app's built-in translation catalog. Use this for any variable whose name matches an app standard token (e.g. `--warn`, `--dark-lum`). The app already supplies EN + DE for these:

```
"--warn": { "default": "#FFB020", "label": "style.var.warn", "group": "accents" }
```

2. Bilingual object - supply both `en` and `de` yourself. Use this for theme-specific names that don't exist in the app catalog:

```
"--value": { "default": "#22CCFF", "label": { "en": "Accent (arc)", "de": "Akzent (Bogen)" }, "g
```

3. Plain string - shown as-is in both languages. Acceptable for purely decorative or brand names that have no translation:

```
"--value": { "default": "#22CCFF", "label": "Arc accent", "group": "accents" }
```

The fallback chain is: `{en, de}` object → key lookup → plain string. A missing `de` key falls back to `en`.

---

10 Security model - no JavaScript

JavaScript does not run in themes. This is enforced in two layers:

Layer 1 - import-time sanitizer (Java, `ThemeManager.sanitizeHtml`). Applied when the ZIP is imported and before the HTML is stored. The following are stripped unconditionally:

- `<script>` tags and their content
- `<iframe>` tags and their content (can embed arbitrary documents)
- `<base>` tags (would change relative URL resolution)
- All inline event handlers (`onclick=`, `onerror=`, `onload=`, etc.)
- `javascript:` URIs in `href`, `src`, `action` and `formaction` attributes
- `data:` URIs in `src` and `href` (can carry HTML or JS payloads)
- External `http://` and `https://` URLs in `src` and `href`

Layer 2 - load-time sanitizer. The same sanitizer runs again when the theme is read from disk and injected into the page, so stored themes that pre-date a sanitizer update are re-checked on every load.

Why scripts would not work even without the sanitizer: `<script>` tags injected via `innerHTML` are not executed by modern browsers and Android WebView. The sanitizer exists as defence-in-depth, not as the only line of protection.

What you can do instead of JavaScript:

- Use `data-lb-state` for conditional CSS classes.
- Use CSS `@keyframes`, transitions and custom properties for animation.
- Use `data-lb-action` for the built-in speed-unlock tap.

---

11 How to import

11.1 Step 1 - Create the ZIP

All three files must be **at the root of the archive** (not inside a subfolder). The app looks for `theme.html`, `theme.css` and `theme.json` by filename regardless of path, so a subfolder also works, but root is cleaner.

Windows: select all three files in Explorer, right-click → "Compress to ZIP file" (Windows 11) or "Send to → Compressed (zipped) folder".

macOS / Linux terminal:

```
zip my-theme.zip theme.html theme.css theme.json
```

Android (for quick testing): use a file manager app that supports ZIP creation (e.g. MT Manager, ZArchiver).

11.2 Step 2 - Import

1. Transfer the ZIP to your phone (AirDrop, cable, cloud drive, etc.).
2. In the app: **Settings → Display → Theme manager → Import theme**.
3. Pick the ZIP from your file browser.
4. The theme appears in the list; tap **Activate** to apply it.
5. A **Theme → Style** button opens the Style Editor, where the user can override colours, brightness and font sizes on top of your theme.

To export a theme you already imported (for sharing or backup): **Theme manager → (tap the theme) → Export**.

---

12 Portability between Laufbursche apps

The ZIP bundle format and the `data-lb` attribute names are a **shared API across all Laufbursche apps**. A theme built for one app can be imported into any other Laufbursche app that supports custom themes.

When building a theme meant to be used across multiple apps:

- Use only the `data-lb` keys listed in Available data keys - these are the stable cross-app set.
- Use only the CSS variables listed in App-provided CSS variables.
- Do not rely on fixed IDs or classes that belong to one app's own HTML (like `#speed-row` from the default layout).

Apps may expose additional app-specific keys or CSS variables beyond this list; check each app's own documentation.

---

13 Translations and language

The app runs in German or English. When you switch languages the app re-runs the full i18n pass over the entire document, including your injected theme HTML.

13.1 Using existing app strings

Add `data-i18n="key"` to any element and the app replaces its text content with the translated string at runtime. The fallback text between the tags is shown in a browser during development:

```
<span data-i18n="dash.hero.gear.label">Gear</span>
```

You can only reference keys that already exist in the app's built-in i18n table. Useful built-in keys for theme elements:

Key	DE	EN
`dash.hero.gear.label`	Gang	Gear
`dash.hero.battery.unit`	Batterie %	Battery %
`dash.motors.section`	Motoren	Motors
`dash.motor.rear.title`	HINTEN	REAR
`dash.motor.front.title`	VORNE	FRONT
`dash.motor.power.title`	LEISTUNG	POWER
`dash.speed.vcu.unit`	km/h - VCU	km/h - VCU
`dash.speed.gps.unit`	km/h - GPS	km/h - GPS

`data-i18n-aria`, `data-i18n-title` and `data-i18n-alt` work the same way for `aria-label`, `title` and `alt` attributes.

13.2 Adding your own translated strings

Declare a `strings` block in `theme.json`. The app merges these into the global i18n table when your theme loads, so they work exactly like built-in keys:

```
{
  "name": "My Theme",
  "version": "1.0",
  "strings": {
    "de": {
      "mtheme.speed.label": "Tempo",
      "mtheme.gear.label":  "Gang",
      "mtheme.unit.kmh":    "km/h"
```

```
    },
    "en": {
      "mtheme.speed.label": "Speed",
      "mtheme.gear.label":  "Gear",
      "mtheme.unit.kmh":    "km/h"
    }
  }
}
```

Then in `theme.html`:

```
<span class="lbl" data-i18n="mtheme.speed.label">Speed</span>
<span class="val" data-lb="speed">-</span>
<span class="unit" data-i18n="mtheme.unit.kmh">km/h</span>
```

Key naming: prefix your keys with something unique to your theme (e.g. the theme name) to avoid collisions with built-in keys or other themes. A missing key falls back to the fallback text you wrote between the tags, so the interface stays readable even if a key is undefined.

One language only: if your theme targets a single audience, just hardcode the labels directly in the HTML without `data-i18n`. That is perfectly fine and keeps the bundle simpler.

---

14 Complete example - Orbit (full-scope teaching theme)

Orbit is a complete, importable theme that demonstrates every technique in this guide: a circular SVG speed gauge, `lb-unlocked` scale extension, `lb-mph` unit toggle, all three transparency vars wired, a data grid, a button with an inline SVG icon, and a park state indicator. Save the three files below, zip them to `Orbit.zip`, and import directly.

14.1 theme.json

```
{
  "name": "Orbit",
  "version": "1.0",
  "scope": "full",
  "cssVars": {
    "--value":     "#BCEB39",
    "--btnfill":   "#BCEB39",
    "--dark-card": "#111420",
    "--dark-bg":   "#0A0C10"
  }
}
```

14.2 theme.html

```
<!-- ORBIT - scope:"full" replaces #scroller (entire dashboard scroll area).
     Three data-tile-id sections let the user show/hide/reorder each block.
     All class names are prefixed ot- to avoid collisions with app styles. -->

<!-- HERO: speed gauge + SOC row.
```

```
     opacity: var(--tile-alpha) in theme.css wires the Kachel-Transparenz slider. -->
<div class="ot-hero" data-tile-id="ot-hero" data-tile-label="Tacho">

  <!-- Inline SVG gauge - no external images or fonts.
       viewBox "0 0 160 160"; CSS sets rendered size.
       Circumference at r=58: ~364 px. 270° arc = 273 px; gap = 91 px.
       rotate(135 80 80): start at 7 o'clock instead of 3 o'clock. -->
  <svg class="ot-gauge" viewBox="0 0 160 160" aria-hidden="true">

    <!-- Track ring: static 270° grey background arc. -->
    <circle class="ot-track"
      cx="80" cy="80" r="58" fill="none"
      stroke-dasharray="273 91"
      transform="rotate(135 80 80)" />

    <!-- Fill arc: stroke-dashoffset calculated in CSS from --cur-speed.
         --gauge-alpha wires the Tacho-Transparenz slider (see theme.css). -->
    <circle class="ot-arc"
      cx="80" cy="80" r="58" fill="none"
      stroke-dasharray="273 91"
      transform="rotate(135 80 80)" />

    <!-- data-lb="speed" wires the live km/h value; app sets textContent every 500 ms. -->
    <text class="ot-speed-val" x="80" y="76" text-anchor="middle"
          data-lb="speed">-</text>

    <!-- Unit labels: two overlapping <text> elements toggled by CSS opacity.
         display:none is unreliable on SVG text in Android WebView. -->
    <text class="ot-unit ot-unit-kmh" x="80" y="97" text-anchor="middle">km/h</text>
    <text class="ot-unit ot-unit-mph" x="80" y="97" text-anchor="middle">mph</text>

    <!-- Lock badge: same opacity toggle technique. -->
    <text class="ot-badge ot-badge-locked"   x="80" y="126" text-anchor="middle">LOCKED</text>
    <text class="ot-badge ot-badge-unlocked" x="80" y="126" text-anchor="middle">UNLOCKED</text>
  </svg>

  <!-- SOC row. data-lb-state adds ot-park-on when scooter is parked. -->
  <div class="ot-soc-row">
    <span class="ot-soc-val" data-lb="soc">-</span><span class="ot-soc-unit">%</span>
    <span class="ot-park" data-lb-state="park:ot-park-on">P</span>
  </div>
</div>

<!-- DATA GRID -->
<div class="ot-grid" data-tile-id="ot-grid" data-tile-label="Daten">
  <div class="ot-cell">
    <span class="ot-lbl">Volt</span>
    <span class="ot-val" data-lb="volt">-</span>
    <span class="ot-usm">V</span>
```

```
  </div>
  <div class="ot-cell">
    <span class="ot-lbl">Strom</span>
    <span class="ot-val" data-lb="current">-</span>
    <span class="ot-usm">A</span>
  </div>
  <div class="ot-cell">
    <span class="ot-lbl">Gang</span>
    <span class="ot-val" data-lb="gear">-</span>
  </div>
  <div class="ot-cell">
    <span class="ot-lbl">Trip</span>
    <span class="ot-val" data-lb="singleMile">-</span>
    <span class="ot-usm">km</span>
  </div>
</div>

<!-- BUTTON ROW.
     data-lb-action="speed-unlock" wires the built-in triple-tap unlock.
     Icon uses fill="currentColor" to inherit the CSS color property. -->
<div class="ot-buttons" data-tile-id="ot-buttons" data-tile-label="Buttons">
  <div class="ot-btn" data-lb-action="speed-unlock">
    <svg class="ot-btn-icon" viewBox="0 0 24 24" fill="currentColor" aria-hidden="true">
      <path d="M7 2v11h3v9l7-12h-4l4-8z"/>
    </svg>
    <span class="ot-btn-lbl">Unlock</span>
  </div>
</div>
```

14.3 theme.css

```
/* === ORBIT  -  ot- prefix keeps all names unique to this theme === */

/* --- HERO ------------------------------------------------------------
   opacity: var(--tile-alpha) wires the "Kachel-Transparenz" slider.
   --tile-alpha is a 0-1 float (1 = fully opaque). */
.ot-hero {
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 16px 12px 8px;
  background: var(--card);
  border-radius: var(--radius);
  margin-bottom: 8px;
  opacity: var(--tile-alpha);
}

/* Gauge SVG: fills container up to 200 px.
   overflow:visible lets the stroke paint outside the viewBox edge. */
```

```
.ot-gauge { width: 100%; max-width: 200px; overflow: visible; }

/* Track ring: static background arc. */
.ot-track {
  stroke: var(--border);
  stroke-width: 10;
  stroke-linecap: round;
}

/* Fill arc: driven by --cur-speed.
   --speed-max: 25 = eKFV limit; extends to 110 when lb-unlocked.
   stroke-dashoffset = 273 (arc hidden) at speed 0,
                     = 0   (full arc) at speed == --speed-max.
   --gauge-alpha wires the "Tacho-Transparenz" slider. */
.ot-arc {
  --speed-max: 25;
  stroke: rgb(from var(--value) r g b / var(--gauge-alpha));
  stroke-width: 10;
  stroke-linecap: round;
  stroke-dashoffset: calc(273 - var(--cur-speed) * 273 / var(--speed-max));
  transition: stroke-dashoffset 0.4s ease-out;
}
:root.lb-unlocked .ot-arc { --speed-max: 100; }

.ot-speed-val { font-size: 38px; font-weight: var(--fw-bold); fill: var(--text); font-family: in

/* Unit labels: opacity toggle - display:none is unreliable on SVG text
   in Android WebView. */
.ot-unit       { font-size: 12px; fill: var(--label); font-family: inherit; }
.ot-unit-kmh   { opacity: 1; }
.ot-unit-mph   { opacity: 0; }
:root.lb-mph .ot-unit-kmh { opacity: 0; }
:root.lb-mph .ot-unit-mph { opacity: 1; }

/* Lock/unlock badge: same opacity technique. */
.ot-badge            { font-size: 9px; letter-spacing: 0.08em; font-family: inherit; }
.ot-badge-locked     { fill: var(--warn);   opacity: 1; }
.ot-badge-unlocked   { fill: var(--value);  opacity: 0; }
:root.lb-unlocked .ot-badge-locked   { opacity: 0; }
:root.lb-unlocked .ot-badge-unlocked { opacity: 1; }

.ot-soc-row  { display: flex; align-items: baseline; gap: 4px; margin-top: 6px; }
.ot-soc-val  { font-size: 22px; font-weight: var(--fw-bold); color: var(--value); }
.ot-soc-unit { font-size: var(--fs-sm); color: var(--label); }

/* Park badge: hidden until ot-park-on is set by data-lb-state. */
.ot-park          { display: none; font-size: var(--fs-xs); color: var(--warn); font-weight: var
.ot-park.ot-park-on { display: inline; }
```

```css
/* --- DATA GRID ------------------------------------------------
   opacity: var(--tile-alpha) shares the Kachel-Transparenz slider. */
.ot-grid {
  display: flex;
  gap: 6px;
  margin-bottom: 8px;
  opacity: var(--tile-alpha);
}
.ot-cell {
  flex: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 8px 4px;
  background: var(--card);
  border-radius: var(--radius);
}
.ot-lbl { font-size: var(--fs-xs); color: var(--label); }
.ot-val { font-size: 20px; font-weight: var(--fw-bold); color: var(--text); }
.ot-usm { font-size: var(--fs-xs); color: var(--label); }

/* --- BUTTON ROW -----------------------------------------------
   opacity: var(--btn-alpha) wires the "Button-Transparenz" slider. */
.ot-buttons { display: flex; gap: 8px; padding-bottom: 4px; }
.ot-btn {
  flex: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 4px;
  padding: 12px 8px;
  background: var(--btnfill);
  border-radius: var(--radius);
  cursor: pointer;
  opacity: var(--btn-alpha);
}
/* fill="currentColor" on the SVG picks up this color. */
.ot-btn-icon { width: 24px; height: 24px; color: var(--bg); }
.ot-btn-lbl  { font-size: var(--fs-xs); font-weight: var(--fw-bold); color: var(--bg); }
```

---

15 Tips and limitations

- **`innerHTML` injection.** Your HTML replaces the entire content of `#fold-inner` (fold scope) or `#scroller` (full scope). The app re-injects it on every theme activation and page reload. Do not reference IDs from the default layout (like `#speed-row`) - they will not exist.
- **No external images.** `http://` and `https://` URLs are stripped from `src` attributes. Use

CSS `background-color` and SVG shapes for icons and graphics.

- **No custom fonts from the web.** Stick to the system font stack or inline a base64 font-face (keep file size in mind - 2 MB limit).
- **Default scope covers only the fold.** Your theme replaces `#fold-inner`. Use `"scope":"full"` in `theme.json` to also replace the scroll area (see Theme scope).
- **CSS load order.** `theme.css` is loaded as `active-theme.css` right after `dashboard.css` in the page `<head>`. Your selectors win over app rules at equal specificity. Use your own class names (prefix them) to avoid accidentally changing app UI outside `#fold-inner`.
- **Content Security Policy.** The page enforces `style-src file:` - only local files are allowed. External CSS, Google Fonts and icon CDNs are blocked at the browser level before the sanitizer even runs. Inline font-face with base64 data works; external font URLs do not.
- **Dark / light mode.** Do not hardcode colours - use `var(--card)`, `var(--text)`, etc. so the theme works in both modes automatically.
- **Testing.** Open `telemetry.html` directly in a desktop browser (Chrome / Firefox). The `LB` bridge is absent so the theme CSS and HTML do not load from storage, but you can: (a) paste your HTML into `#fold-inner` via DevTools, (b) manually inject a `<link>` pointing to your local `theme.css`, (c) add a `<script>` that fills `data-lb` elements with dummy values - that script is only for browser testing and must be removed before zipping.
- **Tile manager sync.** If you add `data-tile-id` elements, their labels appear in the theme manager. Keep labels short (under 20 characters) so the list stays readable.
- **`rgb()` relative colour syntax: flat origins only.** `rgb(from var(--card) r g b / var(--tile-alpha))` works on Android WebView 111+. `rgb(from color-mix(...) r g b / alpha)` does **not** - a `color-mix()` origin silently drops the alpha channel. Use a flat CSS variable as the origin (e.g. `var(--card)`, `var(--value)`, `var(--btnfill)`), never a `color-mix()` result.